

实践3：人形机器人动作空间 HoST Sim2Sim 部署



深蓝学院《人形机器人运动控制》课程第1章

课程链接：<https://www.shenlanxueyuan.com/course/864>

1. 前置要求

已完成实践 1 中 MuJoCo + PyTorch 环境配置

作业预计用时 2-4 小时

作业核心文件为 `deploy_mujoco_host_student.py`

2. 作业目标

本作业对应课程 **第 2 章第 2 节：人形机器人动作空间（Action Space）**。你需要在 **MuJoCo** 仿真环境中，为 **23 自由度 Unitree G1** 人形机器人部署 **HoST** 预训练策略，完成 **Sim2Sim** 站起控制。

运行入口（完成 TODO 后）：

代码块

```
1 python deploy_mujoco_host_student.py
```

完成本作业后，你应当能够：

能力项	说明
理解增量动作空间设计	掌握 HoST 增量动作空间与标准残差动作空间的设计差异及适用场景
读取 MuJoCo 状态	从 <code>mj_data.qpos</code> / <code>mj_data.qvel</code> 中正确提取浮动基座与 23 个关节的状态
构建 HoST 观测	

	组装 76 维单步观测 <code>o_t</code> ，并维护 6 步历史 <code>o_t</code> （共 456 维）
部署预训练策略	在 MuJoCo 中运行 TorchScript 策略，实现 G1 从躺倒姿态 自主站起
录制仿真视频	通过配置文件开启视频保存，用于作业提交

你需要完成的代码： `deploy_mujoco_host_student.py` 中的 **4 个 TODO**（其余框架代码已提供，请勿修改）。4 个 TODO 分别对应下文中的四个理论环节——状态读取、观测构建、历史维护、动作映射——将在后续展开具体实现要求。

3. 作业背景

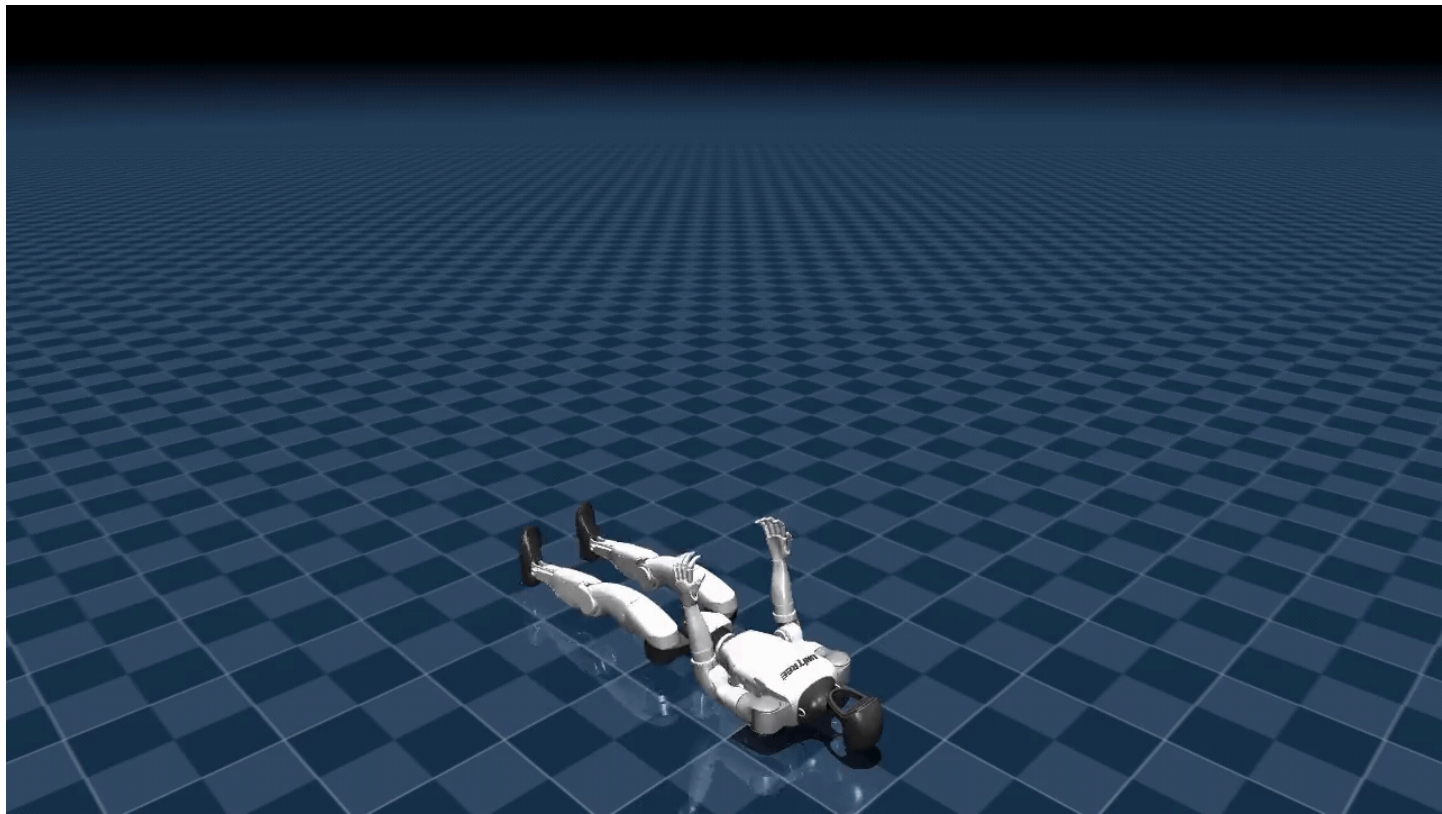
本节完整介绍 HoST 方法的理论背景，内容对应课程讲义 **第 2 章第 2 节**。请先通读作业背景部分，再阅读后续的环境配置与代码实现部分——理论中的每个公式，都将在后续的 TODO 中有直接的代码对应。

符号说明：下文公式均用纯文本表示，变量与代码/config 的对应关系见各节表格。

3.1 任务概述

在 MuJoCo 中加载 **Unitree G1（23 自由度）** 模型，并部署 HoST 预训练策略。机器人初始处于 **侧躺 / 非站立** 姿态；策略以 50 Hz 频率输出关节目标，底层 **PD 控制器** 将其转化为力矩，驱动机器人 **自主站起** 并保持稳定。

运行成功后，MuJoCo 窗口中可看到 G1 从地面姿态逐渐站起，效果如下：



3.2 HoST 方法简介

HoST (Learning Humanoid Standing-up Control across Diverse Postures) 是 RSS 2025 **Best System Paper Finalist**, 专门解决人形机器人在 **多种初始姿态** 下自主站起的问题。本作业的核心目标是理解并实现一种**不同于常规残差形式的动作空间设计: 增量动作空间**, 即策略输出相对于 **当前关节状态** 的增量, 而非相对于固定标称姿态的偏移。你将在仿真中完成该设计从理论到代码的 sim2sim 部署。

3.3 观测空间：本体感知与 POMDP

HoST 仅使用 **本体感知 (Proprioceptive)** 信息, 不依赖相机等外部传感器。单步观测 `o_t` (76 维) 定义为:

式 (1)

代码块

```
1  o_t = [  $\omega_t^{gb}$ , g_t, q_t, q_{dot_t}, a_{\{t-1\}}, s_a ] (维度: 76)
```

符号	代码/配置对应	维度	含义
<code>ω_t^{gb}</code>	<code>base_angular_velocity_obs</code>	3	基座角速度 (base frame, 已缩放)
<code>g_t</code>	<code>projected_gravity</code>	3	投影重力方向 (base frame)

q_t	joint_positions_for_obs	23	关节位置（已缩放）
q_dot_t	joint_velocities_for_obs	23	关节速度（已缩放）
a_{t-1}	previous_policy_action	23	上一时刻策略输出
s_a	action_scale（来自 gl.yaml）	1	动作缩放系数

站起任务具有 **部分可观测性（POMDP）** —— 单帧观测不足以推断完整状态（例如无法直接感知地面接触情况）。因此策略的输入不是 `o_t`，而是 **最近 6 步观测的拼接**：

式 (2)

代码块

```
1  O_t = [ o_t,  o_{t-1},  o_{t-2},  o_{t-3},  o_{t-4},  o_{t-5} ]      (维度：456
    = 76 × 6)
```

其中 `o_t` 按式 (1) 构建，`s_a` 作为标量嵌入每帧 `o_t` 的末位。策略网络接收 `O_t`，输出当前动作 `a_t`。

代码对应：

式 (1) → TODO 2 `build_single_observation()`；

式 (2) → TODO 3 `update_observation_history()`。

3.4 控制架构：策略输出与 PD 控制

在 HoST 中，策略输出的是 **关节空间动作 `a`**，而非直接输出力矩。动作首先被映射为 **关节位置目标 `q*`**，再由底层 **PD 控制器** 计算执行力矩：

式 (3)

代码块

```
1  τ = Kp * (q* - q) + Kd * (q_dot* - q_dot)
```

其中：

- `q`：当前关节角
- `q_dot`：当前关节角速度
- `q_dot* = 0`：目标关节速度恒为零
- `Kp`、`Kd`：PD 增益，已在 `configs/gl.yaml` 中配置，无需修改

关键问题：动作 a 如何映射为 q^* ？这引出了两种截然不同的动作空间设计。**本作业重点介绍并实现其中第二种——增量动作空间**，它与常见的残差动作空间在基准选取与控制行为上有本质区别。

3.5 两种动作空间对比

方式 1：标准残差动作空间（Standard Joint Residual）

以 **固定标称姿态** q_{ref} （Nominal Posture，即 `default_angles`）为基准：

式 (4a) — 关节目标

代码块

$$1 \quad q^* = q_{\text{ref}} + s_a * a$$

式 (4b) — 代入 PD 后的力矩

代码块

$$1 \quad \tau = K_p * (q_{\text{ref}} - q + s_a * a) - K_d * \dot{q}$$

特性分析：

- PD 控制项 $K_p * (q_{\text{ref}} - q)$ 始终存在，持续将关节拉向 q_{ref}
- 即使 $a = 0$ ，仍有恢复力 $\tau \approx K_p * (q_{\text{ref}} - q)$ ，机器人被"锚定"在标称姿态附近
- 策略的探索空间受 q_{ref} 约束：当 q 远离 q_{ref} 时（如侧躺地面）， $|q_{\text{ref}} - q|$ 很大，PD 会产生强烈拉回力矩，**不利于 contact-rich 的站起动作**

方式 2：HoST 增量动作空间（Incremental Action Space）——本作业实现对象

以 **当前关节位置** q （即 `q_current`）为基准。这是本作业所采用的动作空间设计：

式 (5a) — 关节目标

代码块

$$1 \quad q^* = q + s_a * a$$

式 (5b) — 代入 PD 后的力矩

代码块

$$\begin{aligned} 1 \quad \tau &= K_p * (q + s_a * a - q) - K_d * \dot{q} \\ 2 \quad &= K_p * (s_a * a) - K_d * \dot{q} \end{aligned}$$

特性分析：

- 消去了 $(q_{ref} - q)$ 项——**不再依赖标称姿态**
- 当 $a = 0$ 时， $q^* = q$ ，PD 仅输出阻尼项 $-K_d * \dot{q}$ ，机器人不会被拉向任何固定姿态
- 策略输出的是相对于 **当前姿态** 的增量（Delta），天然适合从任意初始姿态（侧躺、俯卧等）逐步调整至站立

关键理解：g1.yaml 中的 default_angles 仅用于 仿真初始化，不是 式 (5a) 中的 q 。动作映射的基准始终是 当前时刻 的关节角 $q_{current}$ 。

代码对应：

式 (5a) → TODO 4 action_to_joint_targets()；

式 (1) 中的 q_t 、 ω_t^{gb} 等 → TODO 1 read_robot_state()。

两种动作空间对比一览

	标准残差（式 4a）	HoST 增量（式 5a，本作业）
动作基准	固定 q_{ref}	当前 $q_{current}$
$a = 0$ 时	被拉向 q_{ref}	保持当前姿态

3.6 增量动作空间的设计意义

理解式 (5) 与式 (4) 的差异，不仅是为了完成本作业，更关系到人形机器人控制的一种通用设计思路：

1. 摆脱 Motion Priors

传统站起 / 起身方法常依赖 **Motion Priors**——需要预先提供参考动作轨迹或关键帧姿态 q_{ref} 。HoST 通过增量动作空间，使策略在 **无先验（Prior-free）** 条件下也能从多种姿态自主站起。

2. 适配 Contact-rich 任务

站起过程中，机器人与地面持续接触，动力学高度非线性。增量动作空间让策略逐步输出 **相对当前状态的微调**，而非被拉向一个固定的标称姿态，更适合这类 **接触丰富（contact-rich）** 的任务。

3. 向高层策略延伸

增量动作空间的本质是：策略输出 **当前状态的 Delta**，而非绝对目标。这一思路可以延伸到 **VLA（Vision-Language-Action Model）** 等高层决策系统——语言指令给出 task-level 目标，底层控制器以 **增量** 方式执行，两者自然解耦。

3.7 从理论到代码

以上理论在代码中形成如下闭环：

代码块

```
1 MuJoCo 仿真状态
2   → [TODO 1] 读取  $q, \dot{q}, \omega, g$  (式 1 的输入来源)
3   → [TODO 2] 构建  $o_t$  (76 维) (式 1)
4   → [TODO 3] 维护  $o_t$  (456 维) (式 2)
5   → 预训练策略推理 →  $a_t$ 
6   → [TODO 4] 计算  $q^* = q + s_a * a$  (式 5a)
7   → PD 控制器 →  $\tau$  (式 3)
8   → MuJoCo 物理步进
```

接下来，将介绍环境配置与代码结构，以及给出 4 个 TODO 的详细实现要求。

4. 环境配置

完成上述理论阅读后，按以下步骤准备运行环境。

4.1 前置条件

本作业假设你已在 **实践 1** 中完成以下环境配置：

- Python 3.8+
- MuJoCo (含 `mujoco.viewer` 交互窗口)
- PyTorch (用于加载 `.pt` TorchScript 策略)

如尚未配置，请先完成实践 1 的环境搭建，再开始本作业。

4.2 本作业额外依赖

根据 `deploy_mujoco_host_student.py` 的 import 及作业提交要求，本作业需要以下 Python 包：

包名	用途
<code>mujoco</code>	MuJoCo 物理仿真与可视化
<code>numpy</code>	数组运算
<code>torch</code>	加载与运行 TorchScript 策略
<code>pyyaml</code>	读取 <code>configs/g1.yaml</code>
<code>imageio</code>	录制提交用仿真视频 (<code>outputs/simulation.mp4</code>)
<code>imageio-ffmpeg</code>	<code>imageio</code> 写 MP4 的后端依赖

以上均为本作业 **必需** 依赖。其中 `mujoco`、`numpy`、`torch` 应已在实践 1 中安装；本作业需额外安装 `pyyaml`、`imageio`、`imageio-ffmpeg`。

4.3 安装命令

在实践 1 的 conda / venv 环境中执行：

代码块

```
1 pip install pyyaml imageio imageio-ffmpeg
```

如需从头确认全部依赖，可执行：

代码块

```
1 pip install mujoco numpy torch pyyaml imageio imageio-ffmpeg
```

4.4 获取作业代码

1. 从 深蓝学院《人形机器人运动控制》课程第 2 章 下载 `shenlan_hw2_action_space.zip`；
2. 解压到本地目录，例如：

代码块

```
1 unzip shenlan_hw2_action_space.zip
2 cd shenlan_hw2_action_space
```

3. 确认目录结构完整（见下文）

4.5 快速自检

代码块

```
1 python -c "import mujoco, numpy, torch, yaml, imageio; print('环境 OK!')"
```

无报错即可开始编写 TODO。

5. 代码库结构

解压 `shenlan_hw2_action_space.zip` 后，目录结构如下：


```

1 shenlan_hw2_action_space/
2 |— deploy_mujoco_host_student.py  # 学员需完成的入口脚本（4 个 TODO）
3 |— configs/
4 |   |— g1.yaml                    # 仿真与策略配置（PD 增益、缩放系数、路径等）
5 |— policies/
6 |   |— pretrained_humanoid_standup.pt  # HoST 预训练 TorchScript 策略（勿修改）
7 |— robots/
8 |   |— g1/
9 |       |— g1_23dof.xml          # G1 23-DOF MuJoCo 模型
10 |       |— meshes/              # 机器人 STL 网格文件
11 |— assets/
12 |   |— chap2_demo1.gif          # 演示动画（文档用）
13 |— outputs/                     # 运行后自动生成，存放 simulation.mp4

```

5.1 各文件职责

文件	说明
<code>deploy_mujoco_host_student.py</code>	主程序：加载模型与策略，运行仿真循环； 你只需实现 4 个函数
<code>configs/g1.yaml</code>	集中管理路径、PD 参数、观测/动作缩放、仿真频率等
<code>robots/g1/g1_23dof.xml</code>	G1 机器人模型；含 浮动基座 （ <code>floating_base_joint</code> ， <code>type= free</code> ）和 23 个驱动关节
<code>policies/pretrained_humanoid_standup.pt</code>	已训练好的站起策略，输入 <code>(1, 456)</code> ，输出 <code>(1, 23)</code>

5.2 关键配置项（`configs/g1.yaml`）

代码块

```

1  # 策略与模型路径（相对路径，程序会自动解析）
2  policy_path: "policies/pretrained_humanoid_standup.pt"
3  xml_path: "robots/g1/g1_23dof.xml"
4
5  # 仿真：1 ms 物理步长，每 20 步运行一次策略 → 50 Hz 控制频率
6  simulation_dt: 0.001
7  control_decimation: 20
8
9  # 观测缩放
10 ang_vel_scale: 0.25

```

```
11  dof_pos_scale: 1.0
12  dof_vel_scale: 0.05
13
14  # HoST 增量动作缩放 (对应公式中的  $s_a$ )
15  action_scale: 0.25
16
17  # 观测维度
18  single_observation_dim: 76
19  history_length: 6
20
21  # 视频录制 (提交作业时建议开启)
22  save_video: true
23  video_path: "outputs/simulation.mp4"
```

注意：default_angles 是初始关节角，不是 HoST 动作空间的 q_{ref} 。

5.3 MuJoCo 状态向量布局

G1 使用 浮动基座，mj_data 中状态布局为：

数组	索引	内容
qpos	[0:3]	基座位置 (x, y, z)
qpos	[3:7]	基座四元数 (w, x, y, z)
qpos	[7:]	23 个关节角度 (rad)
qvel	[0:3]	基座线速度
qvel	[3:6]	基座角速度
qvel	[6:]	23 个关节角速度

本作业的观测与控制 主要使用 `qpos[7:]`、`qvel[3:6]`、`qvel[6:]` 和 `qpos[3:7]`，这些索引将在 TODO 1 中用到。

6. 控制流程概览

第 3.7 部分的理论闭环在 `main()` 中 (无需修改) 具体实现如下：

代码块

```
1  |-----|
2  |  每个物理步 (1 ms)  |
```

```

3 | 1. PD 控制:  $\tau = \text{PD}(q_{\text{target}}, q, \dot{q}) \rightarrow \text{mj\_data.ctrl}$  |
4 | 2. mj_step 推进一步仿真 |
5 | |
6 | 每 control_decimation=20 步 (50 Hz) |
7 | 3. read_robot_state()  $\rightarrow$  TODO 1 |
8 | 4. build_single_observation()  $\rightarrow$  TODO 2 |
9 | 5. update_observation_history()  $\rightarrow$  TODO 3 |
10 | 6. policy(obs_history)  $\rightarrow$  a |
11 | 7. action_to_joint_targets()  $\rightarrow$  TODO 4  $\rightarrow$  q_target |
12 |_____

```

数据流:

```

代码块
1  MuJoCo mj_data
2       $\rightarrow$  read_robot_state()          # 原始状态 + 缩放
3       $\rightarrow$  build_single_observation()    # 76 维 o_t
4       $\rightarrow$  update_observation_history()  # 456 维 O_t
5       $\rightarrow$  TorchScript policy             # 23 维 a_t
6       $\rightarrow$  action_to_joint_targets()     # q_target = q + s_a * a
7       $\rightarrow$  PD 控制器                     #  $\tau$ 

```

7. 作业任务（4 个 TODO）

打开 `deploy_mujoco_host_student.py`，找到 4 处 `raise NotImplementedError(...)`，在对应函数中填入你的实现。每个函数上方已有 docstring 说明返回值与维度，请仔细阅读。

建议顺序：TODO 1 $\rightarrow$ TODO 2 $\rightarrow$ TODO 3 $\rightarrow$ TODO 4

TODO	函数	对应理论
TODO 1	<code>read_robot_state()</code>	式 (1) 中 <code>q_t</code> 、 <code>q_dot_t</code> 、 <code>ω_t^{gb}</code> 、 <code>g_t</code> 的数据来源
TODO 2	<code>build_single_observation()</code>	式 (1): 构建 76 维 <code>o_t</code>
TODO 3	<code>update_observation_history()</code>	式 (2): 维护 456 维 <code>O_t</code>
TODO 4	<code>action_to_joint_targets()</code>	式 (5a): <code>$q^* = q + s_a * a$</code>

TODO 1：读取 MuJoCo 机器人状态

目标文件：`deploy_mujoco_host_student.py`

函数：`read_robot_state(mj_data, dof_pos_scale, dof_vel_scale, ang_vel_scale)`

简要提示：

- MuJoCo 浮动基座占用 `qpos[:7]`、`qvel[:6]`；23 个关节角从 `qpos[7:]` 读取，关节角速度从 `qvel[6:]` 读取
- 基座四元数为 `qpos[3:7]`，顺序 `[w, x, y, z]`；基座角速度为 `qvel[3:6]`
- 对关节位置、关节速度、基座角速度分别乘以传入的 `scale` 参数，得到观测用向量
- 调用文件中已提供的 `get_gravity_orientation()` 计算投影重力
- 返回 5 个量，顺序不可变；其中 `current_joint_positions` 保持**未缩放**，供 TODO 4 使用

验证：运行时不报错，且返回各向量维度分别为 `(23,)`、`(23,)`、`(23,)`、`(3,)`、`(3,)`

TODO 2：构建 76 维单步观测

目标文件：`deploy_mujoco_host_student.py`

函数：`build_single_observation(...)`

简要提示：

- 用 `np.concatenate` 按以下顺序拼接 6 段向量，得到 `shape` 为 `(76,)` 的数组
- 拼接顺序：基座角速度 (3) → 投影重力 (3) → 关节位置 (23) → 关节速度 (23) → 上一时刻动作 (23) → `action_scale` (1)
- `action_scale` 是 Python 标量，需先包装成长度为 1 的数组再拼接

观测布局（总维度 = 76）：

索引	字段	维度
<code>[0:3]</code>	基座角速度	3
<code>[3:6]</code>	投影重力	3
<code>[6:29]</code>	关节位置	23
<code>[29:52]</code>	关节速度	23
<code>[52:75]</code>	上一时刻策略动作	23
<code>[75]</code>	<code>action_scale</code> 标量	1

验证: `current_obs.shape == (76,)`

TODO 3: 更新 6 步滚动观测历史

目标文件: `deploy_mujoco_host_student.py`

函数: `update_observation_history(observation_history, current_obs, single_observation_dim, history_length)`

简要提示:

- 维护一个 FIFO 缓冲区: 每次丢弃最旧的一帧 (前 `single_observation_dim` 个元素), 再将 `current_obs` 追加到末尾
- 可用 `np.concatenate` 实现切片 + 拼接
- 默认配置下输出 shape 应为 `(456,)`, 即 `76 × 6`

示意图:

代码块

```
1  历史缓冲区 (456,) = [ o_{t-5} | o_{t-4} | o_{t-3} | o_{t-2} | o_{t-1} | o_t ]
2                      ←———— 每次丢弃最左侧 76 维 —————→ ←—— 追加新 76 维
                      —→
```

验证: `observation_history.shape == (single_observation_dim * history_length,)`

TODO 4: HoST 增量动作空间映射

目标文件: `deploy_mujoco_host_student.py`

函数: `action_to_joint_targets(policy_action, current_joint_positions, action_scale)`

简要提示:

- 实现式 (5a): `q* = q_current + s_a * a`
- 增量基准是 **当前关节角** `current_joint_positions` (TODO 1 返回的未缩放值), **不是** `default_angles`
- 先计算 `policy_action * action_scale` 得到关节增量, 再添加到当前关节角上
- 返回 shape 为 `(23,)` 的目标关节角

对比 (不要使用):

代码块 $\dot{q} = q_{ref} + s_a * a$ ← 标准残差空间（式 4a），本作业不用

验证：当 `policy_action` 全零时，返回值应等于 `current_joint_positions`

8. 运行与验证

8.1 运行仿真

在 `shenlan_hw2_action_space/` 目录下执行：

代码块

```
1 python deploy_mujoco_host_student.py
```

8.2 预期现象

- 弹出 MuJoCo 交互窗口
- G1 从初始躺倒姿态开始，**逐渐自主站起**
- 站起后保持平衡，仿真持续约 60 秒（可在 `g1.yaml` 中调整 `simulation_duration`）
- 终端无 `NotImplementedError` 或 `shape` 相关报错

8.3 录制提交视频

编辑 `configs/g1.yaml`：

代码块

```
1 save_video: true
2 video_path: "outputs/simulation.mp4"
```

重新运行后，视频保存至 `outputs/simulation.mp4`。

8.4 调试建议

现象	可能原因	排查方向
机器人立刻倒下 / 乱动	TODO 4 用了 <code>default_angles</code> 而非 <code>current_joint_positions</code>	检查增量基准

shape mismatch 报错	观测拼接顺序或维度错误	打印各段 <code>.shape</code> ，对照 76 维布局表
策略加载失败	路径错误或 torch 版本问题	确认 <code>policies/</code> 存在且 <code>torch.jit.load</code> 可用
窗口闪退	MuJoCo viewer 环境问题	确认作业 1 中 viewer 可正常运行
机器人不动	TODO 1/2/3 返回全零或 NaN	打印 <code>current_obs</code> 检查数值范围

推荐调试打印（完成 TODO 后临时添加）：

代码块

```
1 if sim_step_counter == control_decimation:
2     print("obs shape:", current_obs.shape, "history:",
      observation_history.shape)
3     print("action range:", policy_action.min(), policy_action.max())
4     print("target vs current:", np.linalg.norm(target_joint_positions -
      current_joint_positions))
```

9. 提交要求

请将 代码 与 仿真视频 一并打包，命名为 用户名-实践3.zip 提交。

9.1 提交内容

序号	文件	说明
1	deploy_mujoco_host_student.py	完成全部 4 个 TODO 的代码（必需）
2	outputs/simulation.mp4	站起过程仿真录像， <code>save_video: true</code> 运行后生成（必需）

9.2 打包示例

代码块

```
1 # 在 shenlan_hw2_action_space/ 目录下
2 zip 张三-实践3.zip deploy_mujoco_host_student.py outputs/simulation.mp4
```

9.3 提交前检查清单

- ☐ 4 个 TODO 均已实现，无 NotImplementedError
- ☐ 本地运行可看到 G1 自主站起
- ☐ outputs/simulation.mp4 清晰展示站起全过程
- ☐ zip 内同时包含 代码 与 视频 两个文件
- ☐ 压缩包命名符合 张三-实践3.zip 规范

评分标准见评分细则。

10. 评分细则

总分：100 分

本作业按 4 个 TODO 均分 评分，每个 TODO 25 分。助教将依据你提交的 `deploy_mujoco_host_student.py` 代码正确性，以及 `outputs/simulation.mp4` 中 G1 站起效果进行打分。

10.1 分值分配

评分项	分值	对应函数
TODO 1：读取 MuJoCo 机器人状态	25 分	<code>read_robot_state()</code>
TODO 2：构建 76 维单步观测	25 分	<code>build_single_observation()</code>
TODO 3：更新 6 步滚动观测历史	25 分	<code>update_observation_history()</code>
TODO 4：HoST 增量动作空间映射	25 分	<code>action_to_joint_targets()</code>
合计	100 分	—

10.2 各 TODO 评分标准

TODO 1（25 分） — `read_robot_state()`

得分	标准
25 分	正确提取 <code>qpos[7:]</code> 、 <code>qvel[6:]</code> 、 <code>qpos[3:7]</code> 、 <code>qvel[3:6]</code> ；缩放系数应用正确；正确调用 <code>get_gravity_orientation()</code> ；返回值 shape 与类型均正确

15-20 分	主要状态提取正确，但缩放、重力投影或返回值顺序存在 minor 错误
0-10 分	关键索引错误（如混淆浮动基座与关节索引），或函数未实现 / 无法运行

TODO 2（25 分） — `build_single_observation()`

得分	标准
25 分	6 段观测按规范顺序拼接，输出 shape 为 <code>(76,)</code> ， <code>action_scale</code> 正确包装为 1 维数组
15-20 分	拼接逻辑基本正确，但字段顺序或个别维度有误
0-10 分	观测维度错误或拼接顺序与 HoST 定义不一致，导致策略输入异常

TODO 3（25 分） — `update_observation_history()`

得分	标准
25 分	FIFO 滚动逻辑正确，输出 shape 为 <code>(456,)</code> （默认配置），最新帧 appended 在末尾
15-20 分	滚动更新思路正确，但切片边界或 dtype 存在 minor 问题
0-10 分	历史未更新、顺序颠倒（新帧在前），或输出维度错误

TODO 4（25 分） — `action_to_joint_targets()`

得分	标准
25 分	正确使用 HoST 增量公式 <code>q_target = q_current + action_scale * policy_action</code> ；基于 未缩放 的 <code>current_joint_positions</code> ；输出 shape 为 <code>(23,)</code>
15-20 分	增量思路正确，但误用 <code>default_angles</code> 作为基准，或缩放应用有误
0-10 分	仍使用残差动作空间 <code>q_ref + s_a * a</code> ，或函数未实现 / 输出维度错误

10.3 仿真视频与整体效果（扣分项说明）

代码部分满分 100 分，以 4 个 TODO 的独立正确性为主。视频用于 **验证整体部署是否成功**，不作为单独分值项，但可能影响最终认定：

情况	处理
4 个 TODO 均正确，视频展示 G1 成功站起	按上表满分或近满分评定
代码基本正确，但视频未提交	扣 10 分 ；其余按 TODO 分项给分
代码基本正确，视频已提交但机器人未能站起	对应 TODO 酌情扣分（通常问题出在 TODO 3/4）
代码无法运行（含 <code>NotImplementedError</code> ）	对应 TODO 记 0 分

10.4 提交物要求（计分前提）

以下两项 **缺一不可**，缺任一文件视为提交不完整：

1. `deploy_mujoco_host_student.py` — 含 4 个 TODO 的完整实现
2. `outputs/simulation.mp4` — 本地运行生成的站起仿真录像

打包格式：`用户名-实践3.zip`

代码块

```
1 zip 用户名-实践3.zip deploy_mujoco_host_student.py outputs/simulation.mp4
```

11. 常见问题 FAQ

Q1: `default_angles` 和 HoST 动作空间有什么关系？

A: `default_angles` 仅用于仿真 **初始化**，**不是** 式 (4a) 中的 `q_ref`，也不参与式 (5a) 的计算。HoST 动作映射的基准始终是当前关节角 `q_current`。

Q2: 为什么观测里要包含 `action_scale` 和 `previous_policy_action`？

A: 见式 (1)。`a_{t-1}` 和 `s_a` 是 HoST 训练时的观测设计，使策略感知自身动作尺度与上一帧输出，便于 POMDP 下的稳定控制。

Q3: 可以修改 `g1.yaml` 中的 PD 增益吗？

A: 不建议。增益与预训练策略匹配，修改可能导致站起失败。

Q4: `get_gravity_orientation` 需要我实现吗？

A: 不需要，该函数已在 starter 代码中提供，TODO 1 直接调用即可。

Q5: 历史观测的先后顺序重要吗？

A: 重要。式 (2) 要求 `o_t = [o_t, o_{t-1}, ..., o_{t-5}]`，必须保持 **旧** → **新** 排列，最新帧 `current_obs` 在缓冲区 **最右侧**（高索引端），与训练时一致。

12. 参考资料

1. HoST 论文

T. Huang, J. Ren, H. Wang, Z. Wang, Q. Ben, M. Wen, X. Chen, J. Li, and J. Pang,

"Learning Humanoid Standing-up Control across Diverse Postures,"

in Proceedings of the Robotics: Science and Systems (RSS), 2025.

[arXiv:2502.08378](https://arxiv.org/abs/2502.08378)

2. MuJoCo 文档

<https://mujoco.readthedocs.io/>

如有疑问，请在课程群联系助教。祝顺利完成 HoST 部署！